ELEC3441: Computer Architecture
First Semester, 2026–27

Lab 4

Objective: Experiment with Pipeline Operations

In this lab, you will use the Ripes simulator to examine how RISC-V programs move through a five-stage processor pipeline. You will compare forwarding and hazard-detection configurations, observe stalls and incorrect execution, and implement repeated-addition multiplication and iterative Fibonacci functions.

1 Open the Lab Files

Complete the setup and course-update steps in the course repository README before starting. Open a terminal inside the course container and change to:

```
cd /workspace/labs/lab4
ls
```

2 Start Ripes

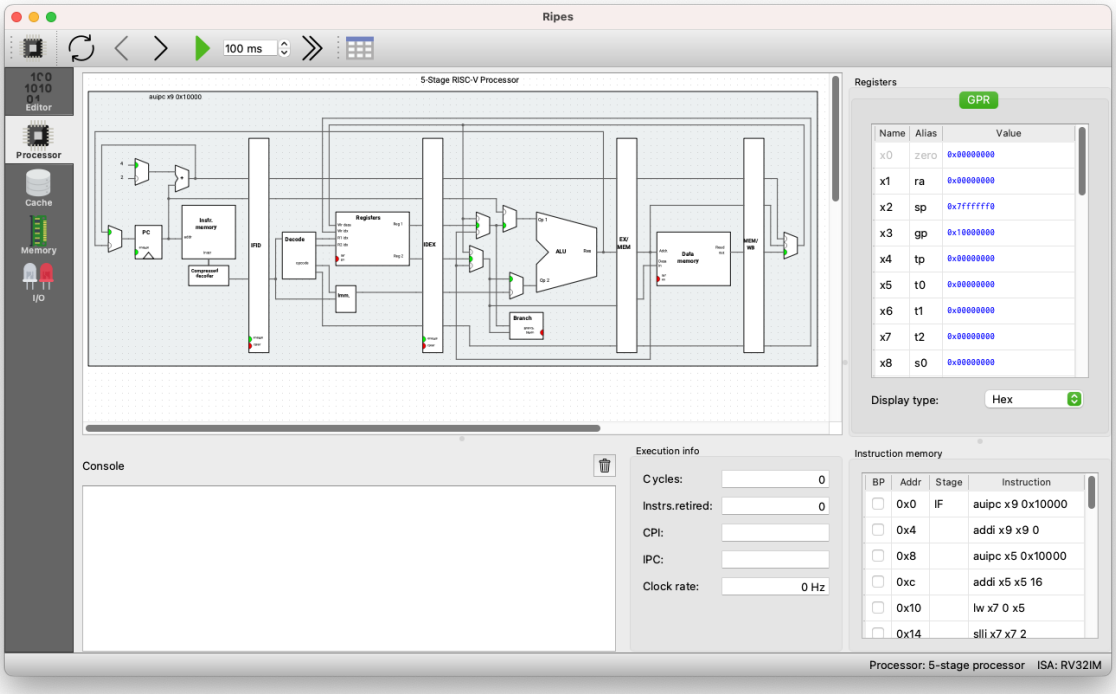
In the terminal inside the course container, run:

```
ripes &
```

Open the browser-desktop address printed by `./manage up`. Ripes should appear in that desktop. The course's pinned Ripes version is already installed in the container; do not download a separate application.

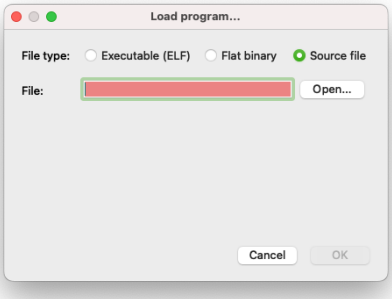
3 Your First Program

3.1 The Ripes Interface The Ripes interface should resemble the screen below. Minor visual differences are expected because the course container uses a newer pinned release.



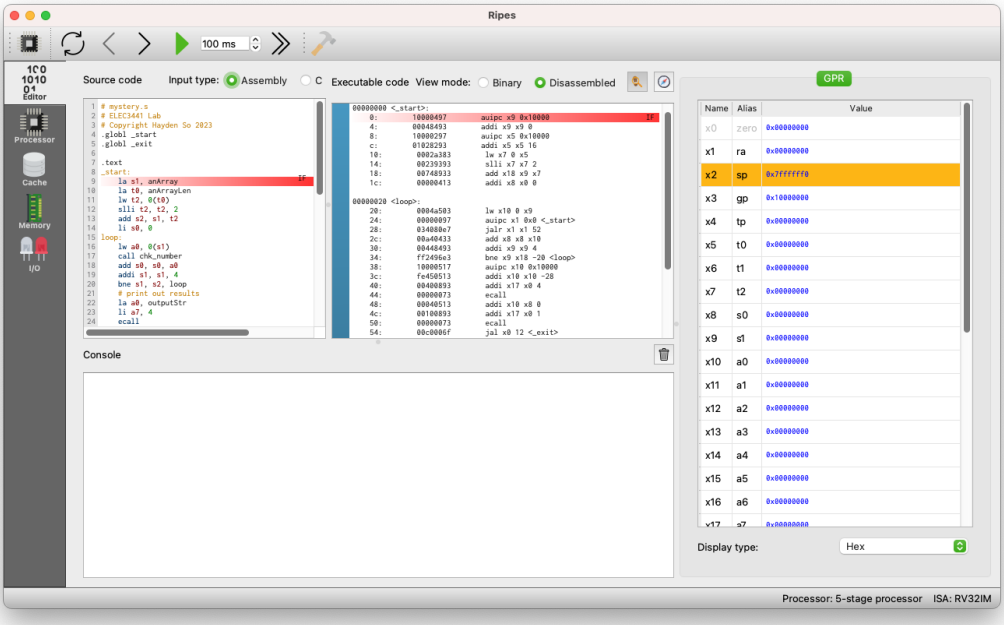
If interface elements are difficult to read, select a light color scheme in the browser desktop.

3.2 Loading Program Use **File** → **Load Program** to load `/workspace/labs/lab4/mystery.s`. This is the same mystery program used in Lab 2. A dialog similar to the one below will appear.



Select **Source File**, then select **Open**... to load the file. Keep the default five-stage processor, which includes forwarding and hazard detection.

3.3 Running the Code The loaded program appears in the **Editor** pane (1C0 1010 01 Editor). This pane shows the source instructions and their assembled machine code:



In the Editor pane, press **Run** (», the double-chevron button) to execute the program to completion. The result appears in the output console. Run does not animate the intermediate pipeline stages. When an exercise asks you to observe pipeline movement, use the single-cycle **Clock** button (>) instead.

Use Run or Clock for this lab; avoid Auto-clock. The supplied Ripes version has an Auto-clock issue that can restart execution after Reset. If you accidentally enable Auto-clock and encounter this behavior, save your program, close and reopen Ripes, and confirm that the cycle count stays at zero.

Switch to the Processor pane (

) to inspect the datapath and pipeline stages.

3.4 Check Yourself

Use **Run** (») to finish the program and inspect the final state in the Editor and Processor panes. Then answer these questions:

(i) What is printed on the output console?
(ii) What is the value stored in the register `a3`?
(iii) How many instructions retire, how many cycles elapse, and what average CPI is displayed? Confirm that `CPI = cycles / retired instructions`.

4 Predict First, Then Observe the Pipeline

Ripes simulates a processor cycle by cycle. In this exercise, first work out what the program should compute, then investigate how pipeline dependencies affect correctness and performance.

4.1 Predict Before Running Load `/workspace/labs/lab4/rtype.s` in the Editor pane. Keep Auto-clock off and **do not run the program yet**. Number the eight instructions I1–I8 in source order. Treat the instructions as executing sequentially, with each instruction seeing the results of earlier instructions.

- Calculate the value written by each instruction. Record your predicted final values of `a0`, `a1`, `a2`, `a3`, and `s0`.
- List each read-after-write (RAW) dependency: which earlier instruction produces a register value that a later instruction reads? Account for intervening overwrites of the same register.

Keep these predictions for comparison with all three processor configurations. Do not change the assembly between runs.

4.2 Run Without Either Mechanism Click **Select Processor** () and choose "5-stage processor w/o forwarding or hazard detection." Reset and confirm that the cycle count stays at zero. If it advances automatically, follow the recovery note in Section 3.3.

Switch to the Processor pane. The first instruction is initially in IF. Press the single-cycle **Clock** button () once: the first instruction moves to ID and the second enters IF. Continue one cycle at a time. Compare the register writes with your predictions, and note when each producer writes back and each consumer reads its operands.

4.3 Checkoff 1

(i) Give your predicted final register values and the RAW dependency list.
(ii) What is the first register write that differs from your prediction? Identify the producing and consuming instructions. Had the producer written back when the consumer read its operand?
(iii) Does the processor wait when an operand is not ready? Explain the final value of `s0`.
(iv) Record retired instructions, cycles, and CPI. Does a low CPI alone establish correct or useful execution?

5 Restore Correctness with Hazard Detection

5.1 Enable Hazard Detection Only Select "5-Stage processor w/o forwarding unit." This model retains hazard detection. Reset and run the *same* `rtype.s` using the single-cycle Clock button.

Hazard detection checks whether a consumer can safely use its operands. When it cannot, the processor holds the consumer and younger instructions and inserts a bubble into EX while older instructions advance. With no forwarding, the consumer must wait for the correct value to become available in the register file.

5.2 Check Yourself

- Observe the dependencies involving `add a1,a0,a0`, `slli a2,a1,8`, and their consumers. Which instructions wait, and where do bubbles appear?
- Record final `s0`, retired instructions, cycles, and CPI. Does the result now match your prediction?
- How many extra cycles are required compared with the previous configuration? Explain how waiting changes correctness and CPI.

Keep these observations for the comparison in Checkoff 2.

6 Reduce Stalls with Forwarding

6.1 Enable Both Mechanisms An ALU result can be available before it is written back to the register file. *Forwarding* routes an available result directly to a dependent instruction, avoiding some waits. It does not eliminate every possible data hazard; hazard detection is still needed when a required value is not yet available.

Select the default "5-stage processor," which includes forwarding and hazard detection. Reset and step through the *same* program. Inspect the forwarding paths when dependent ALU instructions execute, and compare the bubbles with the previous run.

6.2 Checkoff 2

(i) Make a table for the three configurations, in the order tested: neither mechanism, hazard detection only, and both. Include final `s0`, retired instructions, cycles, CPI, and whether the result is correct.
(ii) Explain how hazard detection restores correctness and how forwarding reduces stalls. For this program, how many cycles does forwarding save?
(iii) Assuming equal clock periods, which configuration completes the program correctly in the least time? Why is the equally short incorrect run not a valid alternative?
(iv) Why can CPI exceed 1 even when there are no stalls? Explain pipeline fill and drain.
(v) What is the maximum number of instructions that can occupy this five-stage pipeline simultaneously?

7 Multiplication Function

In this part, you will complete a repeated-addition multiplication function and compare its execution on two processor configurations.

7.1 Use the processor-selection dialog to restore the default "5-stage processor."

7.2 Load `/workspace/labs/lab4/itermult.s`. It contains a skeleton for the function `itermult`:

- The two input values are passed in `a0` and `a1`.
- Return the product in `a0`.
- Do not modify `s0` or `s1`.

A simple way to multiply non-negative values a and b is to add a to a running result b times. For example, $3 \times 4 = 3 + 3 + 3 + 3$. For this exercise, assume that the second operand in `a1` is non-negative. Use temporary registers for the saved multiplicand and loop counter. If you use an unconditional loop back-edge, use `j` rather than `jal` so that the function's return address in `ra` is not overwritten.

7.3 Complete `itermult.s`. Use **Run** (») for each complete test run. Verify at least the cases `(3,4)`, `(7,0)`, `(0,5)`, and `(6,3)` by changing the first two input instructions. Confirm the console result and verify that `s0` and `s1` still hold the original operands.

7.4 Checkoff 3

- Submit evidence that your function produces the correct result for the four test cases above.
- Run the *same implementation* on (1) the default five-stage processor with forwarding and (2) the five-stage processor without forwarding. Record retired instructions, cycles, and CPI for both.
- Assuming equal clock periods, which processor has lower execution time? Explain how forwarding affects stalls.
- Instruction count and CPI depend on your loop structure. Do not compare your CPI directly with another implementation; compare the same source code across processor configurations.

8 Fibonacci Function

You have already studied Fibonacci in Homework 1, Part A, Section 1.4. Use that experience to complete an *iterative* implementation here. The focus of this exercise is to examine its pipeline hazards and compare the same code with and without forwarding.

8.1 Use the processor-selection dialog to restore the default "5-stage processor." Load `fib.s` from the Lab 4 folder. The starter calls a function named `fib` with the following interface:

- The input n, with $0 \le n \le 46$, is passed in `a0`.
- Return $F(n)$ in `a0`, where $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n-1) + F(n-2)$ for $n \ge 2$.
- Implement the function iteratively. Use temporary registers and do not modify `s0`–`s11` or `sp`.

8.2 Complete `fib.s`. Use **Run** (») for each complete test run. Verify `0->0`, `1->1`, `2->1`, `5->5`, and `9->34` by changing the first input instruction. Confirm the console result and verify that `s0` still holds the original input.

8.3 For $n = 9$, run the *same implementation* on (1) the default five-stage processor with forwarding and (2) the five-stage processor without forwarding but with hazard detection. Exact instruction counts and CPI values depend on your loop structure, so compare only the same source code across processor configurations.

8.4 Checkoff 4

- Submit evidence that your function produces the correct result for all five test cases.
- Confirm that `s0` still holds the original input after each call.
- For $n = 9$, record retired instructions, cycles, and CPI on both processor configurations.
- Identify the important RAW dependencies in the Fibonacci loop and explain where the processor without forwarding inserts stalls.
- Assuming equal clock periods, which processor has lower execution time? Why do both produce the same architectural result?

9 Submission

Submit your answers to Checkoffs 1–4, together with your completed `itermult.s` and `fib.s`.